

(c) (25 puntos) Considere la siguiente función de búsqueda de un valor en un arreglo de largo n con extremo izquierdo izq y extremo derecho der :

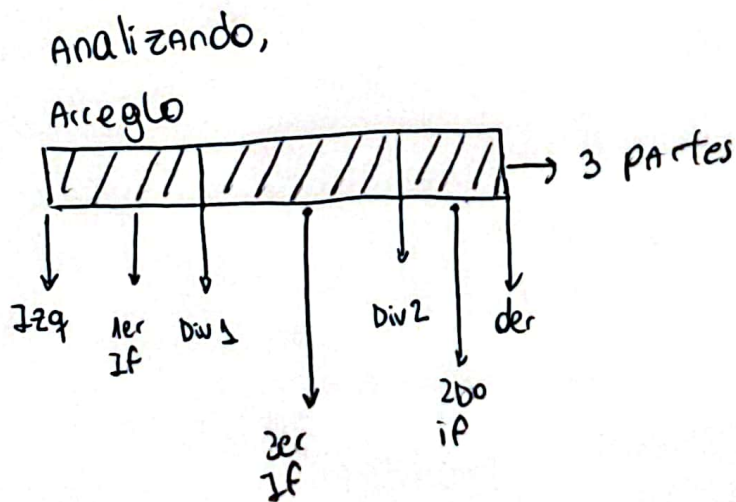
```

1 int BusquedaTernaria(int izq, int der, int valor, int *arreglo){
2   if (der >= izq) {  $\rightarrow T(worst) + \max[T(lf), T(rf)] = 1 + \dots$ 
3      $\rightarrow$  int div1 = izq + (der - izq) / 3;  $\rightarrow 1$ 
4      $\rightarrow$  int div2 = der - (der - izq) / 3;  $\rightarrow 1$ 
5     if (arreglo[div1] == valor) {  $\rightarrow$  Afuera es más costoso, 1
6       return div1;
7     }
8     if (arreglo[div2] == valor) {  $\rightarrow$  Afuera es más costoso, 1
9       return div2;
10    }
11    if (valor < arreglo[div1]) {  $\rightarrow 1$ 
12      return BusquedaTernaria(izq, div1 - 1, valor, arreglo);
13    }
14    else if (valor > arreglo[div2]) {  $\rightarrow 1$ 
15      return BusquedaTernaria(div2 + 1, der, valor, arreglo);
16    }
17    else {  $\rightarrow 1$ 
18      return BusquedaTernaria(div1 + 1, div2 - 1, valor, arreglo);
19    }
20  }
21  return -1;  $\rightarrow 1$ 

```

$T(\frac{n}{3})$

- Determine la función de recurrencia $T(n)$, en función de las instrucciones y la recursividad de la función.
- A partir de $T(n)$, determine la complejidad algorítmica de la función.



Sumando:

$$T(n) = 8 + T\left(\frac{n}{3}\right)$$

$$= O(1) + T\left(\frac{n}{3}\right) //$$

método maestro
 $T(n) = 8 + T\left(\frac{n}{3}\right)$

$$a=1 \quad f(n)=8 \rightarrow O(1)$$

$$b=3$$

$$n^{\log_b a} = n^{\log_3 1} = n^0 = 1.$$

Estamos en caso (2)

$$1 \text{ v/s } 1(\log n)^0 \quad | k=0$$

$$T(n) = \Theta(n^0 \log n)^{k+1}$$

$$T(n) = \Theta(\log n)$$

$$T(n) = O(\log n)$$

$$T(n) = \mathcal{L}(\log n) //$$

$$① t(n) = \tau(\frac{n}{2}) + 2\tau(\frac{n}{3}) + \tau(\frac{n}{4}) + n \log n \quad \tau(1) = 1$$

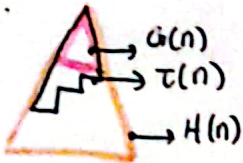
a) Rama más larga

b) Rama más corta

c) complejidades (Ω , Θ , θ)

Paso ②

Como observación:
el arbol nos quedo
asi.



$$G(n) \leq T(n) \leq H(n)$$

① $G(n) = 4t(\frac{n}{4}) + n \log n \rightarrow$ mejor caso por arbol (Ω)

② $H(n) = 4t(\frac{n}{2}) + n \log n \rightarrow$ peor caso por arbol (θ)

① $G(n) = 4t(\frac{n}{4}) + n \log n$

$$\left. \begin{matrix} a=4 \\ b=4 \end{matrix} \right\} n^{\log_4 4} = n^1 =$$

$f(n) = n \log n$
 n^1 v/s $n^1 \log n \rightarrow$ ESTE es CASO ②
POR METODO MAESTRO

$$G(n) = \theta(n \log n)^{k+1} = \theta(n (\log n)^2)$$

$$t(n) = \Omega(n \log n)^2$$

② $H(n) = 4t(\frac{n}{2}) + n \log n$

$$\left. \begin{matrix} a=4 \\ b=2 \end{matrix} \right\} n^{\log_2 4} = n^2$$

$f(n) = n \log n$

n^2 v/s $n^1 \log n \rightarrow$ CASO ①

$$H(n) = \theta(n^2)$$

$$T(n) = \theta(n^2)$$

* Finalmente para Hallar Big θ no se puede encontrar
Pues las complejidades de $G(n)$ y $H(n)$ no coinciden
Es decir, $n(\log n)^2 \neq n^2$.

Apuntes Extra:

Insertsort

$\rightarrow \theta(n^2) \rightarrow$ Todo invertido

$\rightarrow \Omega(n) \rightarrow$ Todo ordenado

$\rightarrow \theta(n^2) \rightarrow$ caso promedio

Mergesort

$\rightarrow \theta(n \log n)$

$\rightarrow \Omega(n \log n)$

$\rightarrow \theta(n \log n)$

Heapsort (Ω, θ)

$\rightarrow \theta(n \log n)$

Quicksort:

$\rightarrow \Omega$ y $\theta(n \log n)$

$\rightarrow \theta(n^2)$

Radix Sort:

$\rightarrow \theta(d(n+k))$

Espacio: $\theta(n+k)$

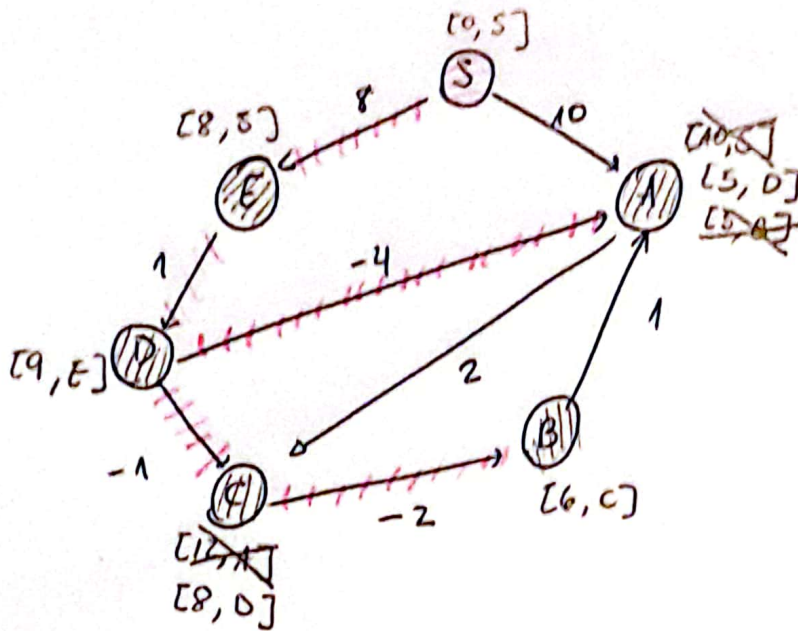
Counting Sort

$\rightarrow \theta(n+k)$

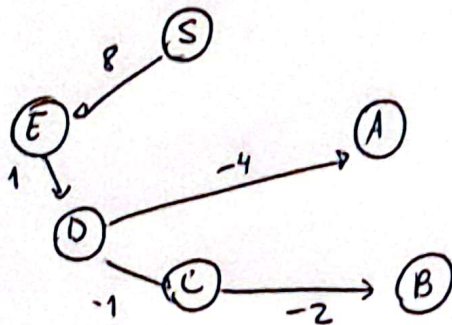
Espacio: $\theta(k)$

Rama más corta
(se divide más rápido) $\rightarrow T(1) = 1$.

Bellman-Ford → Para pesos negativos } algoritmo
 Dijkstra → Para pesos positivos. } Ruta más corta.



algoritmo a cortado:



Reglas:

- ① Comienza en nodo inicial
- ② marcar el que tiene menor distancia
- ③ Actualizar las distancias
- ④ Termina al visitar a todos.